Origin Protocol

Validator Consolidations

Security Assessment Report

Version: 2.0

March, 2026

Contents

Introduction 2
- Disclaimer . . . 2
- Document Structure . . . 2
- Overview . . . 2

Security Assessment Summary 3
- Scope . . . 3
- Approach . . . 3
- Coverage Limitations . . . 3
- Findings Summary . . . 4

Detailed Findings 5

Summary of Findings 6
- `ConsolidationController` Relies On Offchain Trust Assumptions For Correct Accounting . . . 7
- Consolidation Fee Underpayment Silently Drains Consensus Rewards . . . 11
- Permissionless `snapBalances()` Can DoS Consolidation Requests . . . 13
- Excess Fee Payment Misaccounted As Strategy Balance . . . 15
- `MIN_CONSOLIDATION_PERIOD` Underestimates Minimum Consolidation Time . . . 17
- Missing `sourcePubKeys` Input Validation In `requestConsolidation()` . . . 18
- `snapBalances()` Lacks `MIN_CONSOLIDATION_PERIOD` Check . . . 20
- Miscellaneous General Comments . . . 21

A Vulnerability Severity Classification 23

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Origin Protocol components in scope. The review focused solely on the security aspects of the Solidity implementation of the contracts, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Origin Protocol components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see Vulnerability Severity Classification), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Origin Protocol components in scope.

Overview

Origin Protocol is a suite of DeFi products designed to increase economic opportunity through innovative blockchain technologies, including liquid staking tokens and yield-generating stablecoins.

This review covers Origin's validator consolidation mechanism, which migrates OETH collateral from sweeping (`0x01`) validators on the Native Staking Strategies to compounding (`0x02`) validators on the Compounding Staking Strategy. A new `ConsolidationController` contract orchestrates this process, atomically transferring balance accounting between the two strategies.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the OriginProtocol/origin-dollar repository.

The scope of this time-boxed review was strictly limited to the following files at commit 8b86144:

- `contracts/contracts/strategies/NativeStaking/ConsolidationController.sol`
- `contracts/contracts/strategies/NativeStaking/NativeStakingSSVStrategy.sol`
- `contracts/contracts/strategies/NativeStaking/CompoundingStakingSSVStrategy.sol`

The fixes of the identified issues were assessed at commit 57ec469.

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Solidity.

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

To support the Solidity components of the review, the testing team may use the following automated testing tools:

- Aderyn: `https://github.com/Cyfrin/aderyn`
- Slither: `https://github.com/trailofbits/slither`
- Mythril: `https://github.com/ConsenSys/mythril`

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 8 issues during this assessment. Categorised by their severity:

- Medium: 3 issues.
- Low: 1 issue.
- Informational: 4 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Origin Protocol components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each issue by the matrix given in the Appendix: Vulnerability Severity Classification.

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as "informational".

Each vulnerability is also assigned a **status**:

- ***Open:*** the issue has not been addressed by the project team.
- ***Resolved:*** the issue was acknowledged by the project team and updates to the affected components(s) have been made to mitigate the related risk.
- ***Closed:*** the issue was acknowledged by the project team but no further actions have been taken.

Summary of Findings

ID	Description	Severity	Status
OGVC-01	`ConsolidationController` Relies On Offchain Trust Assumptions For Correct Accounting	Medium	Resolved
OGVC-02	Consolidation Fee Underpayment Silently Drains Consensus Rewards	Medium	Resolved
OGVC-03	Permissionless `snapBalances()` Can DoS Consolidation Requests	Medium	Resolved
OGVC-04	Excess Fee Payment Misaccounted As Strategy Balance	Low	Closed
OGVC-05	`MIN_CONSOLIDATION_PERIOD` Underestimates Minimum Consolidation Time	Informational	Resolved
OGVC-06	Missing `sourcePubKeys` Input Validation In `requestConsolidation()`	Informational	Resolved
OGVC-07	`snapBalances()` Lacks `MIN_CONSOLIDATION_PERIOD` Check	Informational	Resolved
OGVC-08	Miscellaneous General Comments	Informational	Resolved

OGVC-01	`ConsolidationController` Relies On Offchain Trust Assumptions For Correct Accounting		
Assets	`contracts/contracts/strategies/NativeStaking/ConsolidationController.sol` `contracts/contracts/strategies/NativeStaking/ValidatorRegistrator.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

The `ConsolidationController`'s consolidation lifecycle relies heavily on the owner (5/8 multisig) and validator registrator to perform correct offchain verification of beacon chain state. Incorrect inputs to any of these functions can cause ETH balance accounting errors in the old Native Staking Strategy.

The old Native Staking Strategy calculates its reported balance in `NativeStakingSSVStrategy.checkBalance()` as:

contracts/contracts/strategies/NativeStaking/NativeStakingSSVStrategy.sol::checkBalance()

```
balance =
    // add the ETH that has been staked in validators
    activeDepositedValidators *
    FULL_STAKE +
    // add the WETH in the strategy from deposits that are still to be staked
    IERC20(WETH).balanceOf(address(this));
```

The `activeDepositedValidators` count is directly reduced by `confirmConsolidation()` in `ValidatorRegistrator`:

contracts/contracts/strategies/NativeStaking/ValidatorRegistrator.sol::confirmConsolidation()

```
function confirmConsolidation(uint256 consolidationCount)
    external
    nonReentrant
    whenNotPaused
    onlyRegistrator
{
    // @audit no verification that consolidations actually completed on beacon chain
    activeDepositedValidators -= consolidationCount;
    // ... snip ...
}
```

This reduction of `activeDepositedValidators` by `consolidationCount` has no onchain verification that the consolidations were actually processed on the beacon chain. The following offchain trust assumptions exist:

1. `requestConsolidation()`: The owner must verify beacon chain activation status.

The contract checks `ValidatorState.ACTIVE` for the target validator, but the code comments explicitly state this does not guarantee actual beacon chain activation:

contracts/contracts/strategies/NativeStaking/CompoundingValidatorManager.sol::verifyBalances()

```
// @audit ACTIVE means balance >= 32.25 ETH, not actual beacon chain activation
} else if (
    validatorDataMem.state == ValidatorState.VERIFIED &&
    validatorBalanceGwei > MIN_ACTIVATION_BALANCE_GWEI
) {
    // Store the validator state as active. This does not necessarily mean the
    // validator is active on the beacon chain yet. It just means the validator has
    // enough balance that it can become active.
    validator[verifiedValidators[i]].state = ValidatorState.ACTIVE;
}
```

A consolidation request will fail silently on the beacon chain if any of the following conditions are not met:

- both source and target validators are active
- neither has initiated an exit
- the source has been active for at least `SHARD_COMMITTEE_PERIOD` epochs
- the source has no pending withdrawals in the queue

If any of these conditions are not met, the consolidation request is silently dropped by the beacon chain. The owner must then detect this offchain and call `failConsolidation()` for the affected validators before calling `confirmConsolidation()`, as described in point 2 below.

2. `failConsolidation()`: The owner must call before `confirmConsolidation()` with all failed validators.

If any source validators fail to consolidate on the beacon chain (e.g. the pending consolidation queue is full, or the source validator was voluntarily exited), the owner must call `failConsolidation()` for every such validator before calling `confirmConsolidation()`. Each call to `failConsolidation()` reduces `consolidationCount`:

contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::failConsolidation()

```
// @audit consolidationCount is reduced for failed validators
consolidationCount -= SafeCast.toUint64(sourcePubKeys.length);
```

If `confirmConsolidation()` is called without first failing all unconsolidated validators, the `consolidationCount` passed to `ValidatorAccountant.confirmConsolidation()` will be larger than the number of validators actually consolidated. This over-reduces `activeDepositedValidators`, undercounting the old strategy's ETH balance by `32 ETH * missed_failures`.

Conversely, if the owner mistakenly calls `failConsolidation()` for a validator that was actually exited via `exitSsvValidator()` (which also sets state to `EXITING`), or for a consolidation request that has yet to be processed by the beacon chain (since `failConsolidation()` has no time-based guard and consolidation requests are irrevocable), this reduces `consolidationCount` without a corresponding reduction in actual consolidated validators. The resulting `confirmConsolidation()` call will under-reduce `activeDepositedValidators`, overcounting ETH.

3. `confirmConsolidation()`: The owner must verify all consolidations have been processed.

The contract checks that `snappedTimestamp > consolidationStartTimestamp + MIN_CONSOLIDATION_PERIOD`, but `MIN_CONSOLIDATION_PERIOD` (256 epochs, ~27 hours) is only a lower bound. Actual processing time depends on the beacon chain's pending consolidation queue and can be significantly longer. If the owner confirms before all consolidations are processed, the target strategy's `verifyBalances()` will report a balance that does not yet include all consolidated ETH, resulting in a temporary undercalculation until the next balance verification.

4. `removeSsvValidator()`: The registrator must verify beacon chain exit offchain.

`ValidatorRegistrator.removeSsvValidator()` allows removal of validators in `EXITING` state but performs no onchain verification that the validator has actually exited the beacon chain:

contracts/contracts/strategies/NativeStaking/ValidatorRegistrator.sol::removeSsvValidator()

```
function removeSsvValidator(
    bytes calldata publicKey,
    uint64[] calldata operatorIds,
    Cluster calldata cluster
) external onlyRegistrator whenNotPaused {
    bytes32 pubKeyHash = keccak256(publicKey);
    VALIDATOR_STATE currentState = validatorsStates[pubKeyHash];
    // @audit only checks contract state, not beacon chain state
    require(
        currentState == VALIDATOR_STATE.EXITING ||
            currentState == VALIDATOR_STATE.REGISTERED,
        "Validator not regd or exiting"
    );
    // ... snip ...
}
```

Removing a validator from the SSV network before it has fully exited the beacon chain results in the validator being unable to perform its duties, resulting in potential inactivity penalties and slashing.

This issue has a high impact as incorrect inputs at any step in the consolidation lifecycle can corrupt the old Native Staking Strategy's reported ETH balance, directly affecting the OETH vault's accounting.

This issue has a low likelihood as it requires operational errors by the trusted 5/8 multisig or validator registrator, both of which are expected to perform careful offchain verification.

Recommendations

Implementing beacon chain state proofs would eliminate all of the offchain trust assumptions described above by allowing the contract to verify validator activation status, exit completion, and consolidation processing onchain. However, beacon chain proofs are expensive and complex to implement, and the additional proof verification logic introduces its own surface area for bugs.

Alternatively, the following smaller onchain checks can reduce the trust surface without the complexity of full beacon chain proofs:

1. **Store source validators onchain (addresses point 2).** When `requestConsolidation()` is called, store the source validator public key hashes in contract storage (e.g. using OpenZeppelin's `EnumerableSet` library). Then `failConsolidation()` can verify that every public key in its input parameters exists in the stored set, preventing the owner from accidentally failing validators that were never part of the consolidation request or that were exited via `exitSsvValidator()`.
2. **Verify strategy balance change in `confirmConsolidation()` (addresses point 2 and 3).** After calling `verifyBalances()` on the target compounding strategy, check that the ETH balance of the strategy has increased by approximately `32 ETH * consolidationCount`. This acts as a sanity check that the consolidations were actually processed on the beacon chain before confirming them. This also partially mitigates the risk from point 2, as an incorrect `consolidationCount` (from missed `failConsolidation()` calls) would be inconsistent with the observed balance change.
3. **Add a time-based guard to `failConsolidation()` (addresses point 2).** Require that at least `MIN_CONSOLIDATION_PERIOD` has elapsed since `requestConsolidation()` before `failConsolidation()` can be called. This ensures the beacon chain has had sufficient time to process or reject the consolidation request, reducing the risk of the owner unwinding execution-layer state for a consolidation that the beacon chain subsequently processes. Keep in mind that `MIN_CONSOLIDATION_PERIOD` is overly-conservative since the actual consolidation request processing time is much shorter.

Resolution

The development team has implemented recommendations **(1)** and **(3)** by adding the following onchain checks to reduce the trust surface area:

- Added a `pendingSourceInRound` mapping to the `ConsolidationController` to store the source validator public key hashes that are pending consolidation in a round. `failConsolidation()` now verifies that every public key in its input parameters exists in the mapping, preventing the owner from accidentally failing validators that were never part of the consolidation request or that were exited via `exitSsvValidator()`.
- Added a minimum delay of `MIN_CONSOLIDATION_PERIOD` to `failConsolidation()` to ensure that the beacon chain has had sufficient time to process or reject consolidation requests.

Recommendation **(2)** was acknowledged by the development team with the following comment:

> *"We did not implement the strategy ETH balance sanity check as validator slashings and withdrawals can make it fail."*

This issue has been resolved in PR #2825.

OGVC-02	Consolidation Fee Underpayment Silently Drains Consensus Rewards		
Assets	contracts/contracts/strategies/NativeStaking/ValidatorRegistrator.sol contracts/contracts/strategies/NativeStaking/ValidatorAccountant.sol contracts/contracts/beacon/BeaconConsolidation.sol		
Status	**Resolved:** See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Consolidation fee underpayment in `requestConsolidation()` silently drains the source strategy's consensus rewards, causing an accounting failure that pauses the contract and deadlocks any in-progress consolidation.

The `ValidatorRegistrator.requestConsolidation()` function calls `BeaconConsolidation.request()` once per source validator. Each call queries the current fee from the EIP-7251 consolidation request system contract and sends that exact amount from `address(this).balance`:

contracts/contracts/beacon/BeaconConsolidation.sol::request()

```
function request(bytes calldata source, bytes calldata target)
    internal
    returns (uint256 fee_)
{
    // ... snip ...
    fee_ = fee();

    // @audit fee is paid from the contract's ETH balance, not from msg.value
    (bool success, ) = CONSOLIDATION_REQUEST_ADDRESS.call{ value: fee_ }(
        abi.encodePacked(source, target)
    );

    require(success, "Consolidation request failed");
}
```

Neither `requestConsolidation()` nor `BeaconConsolidation.request()` verify that `msg.value` covers the total fees. If `msg.value` is less than the cumulative fees (or zero), the shortfall is silently drawn from the contract's existing ETH balance, which includes the tracked `consensusRewards`. Critically, the transaction does not revert, as long as `address(this).balance` exceeds the individual fee at each iteration, the call succeeds without error.

After the call completes, `address(this).balance` has decreased by `(totalFees - msg.value)` while the `consensusRewards` state variable remains unchanged. On the next `doAccounting()` call, the balance invariant check fails:

contracts/contracts/strategies/NativeStaking/ValidatorAccountant.sol::_doAccounting()

```
function _doAccounting(bool pauseOnFail)
    internal
    returns (bool accountingValid)
{
    // @audit balance is now less than consensusRewards due to fee drainage
    if (address(this).balance < consensusRewards) {
        return _failAccounting(pauseOnFail);
    }
    // ... snip ...
}
```

When `_failAccounting()` is called with `pauseOnFail = true` (the default path through `doAccounting()`), it pauses the entire strategy. This blocks all functions protected by `whenNotPaused`, including:

- `confirmConsolidation()`: cannot complete in-progress consolidations
- `failConsolidation()`: cannot abort in-progress consolidations
- `doAccounting()`: cannot process further validator withdrawals or rewards
- `exitSsvValidator()`: cannot exit validators from the beacon chain
- `removeSsvValidator()`: cannot remove validators from the SSV cluster
- `_collectRewardTokens()`: cannot harvest execution or consensus rewards

If a consolidation is in progress when the pause occurs, a deadlock forms: `confirmConsolidation()` and `failConsolidation()` on `ValidatorRegistrator` both require the source strategy to be unpaused, but the strategy cannot unpause until accounting is manually fixed. Meanwhile, `ConsolidationController` enforces `consolidationCount == 0` for new consolidation requests and for `removeSsvValidator()`, so these operations are also blocked until the current consolidation is resolved.

The only recovery path is for the strategist to call `manuallyFixAccounting()` with a negative `_consensusRewardsDelta` to reduce `consensusRewards` to match the decreased balance and unpause the strategy. This is rate-limited by `MIN_FIX_ACCOUNTING_CADENCE` but should suffice given the typically small consolidation fees.

This issue has a medium impact as it pauses the source strategy, deadlocks any in-progress consolidation, and blocks all validator operations and reward harvesting until manually resolved. No direct loss of funds occurs as the fee ETH is legitimately consumed by the system contract, and recovery via `manuallyFixAccounting()` is possible.

This issue has a medium likelihood as the EIP-7251 consolidation fee is dynamic and the multisig introduces an unavoidable delay between transaction preparation and execution, during which the fee can change. Neither `requestConsolidation()` nor `BeaconConsolidation.request()` validate `msg.value` against the actual fee, so underpayment succeeds silently with no revert or error signal to alert the operator.

Recommendations

Verify that `msg.value` covers the total consolidation fees and refund any excess. Track the cumulative fee returned by `BeaconConsolidation.request()` across the loop in `requestConsolidation()`, then require that `msg.value` is at least equal to the total and return any surplus to `msg.sender`.

Resolution

The development team has implemented the recommended change to verify that `msg.value` covers the total consolidation fees.

This issue has been resolved in PR #2819.

OGVC-03	Permissionless `snapBalances()` Can DoS Consolidation Requests		
Assets	`contracts/contracts/strategies/NativeStaking/ConsolidationController.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Consolidation requests can be indefinitely blocked by any external caller frontrunning `requestConsolidation()` with a call to `snapBalances()`, preventing the owner from initiating validator consolidations.

The `ConsolidationController.snapBalances()` function is permissionless when no consolidation is in progress (`consolidationCount == 0`). It forwards the call to `CompoundingValidatorManager.snapBalances()`, which enforces a 420-second cooldown (`SNAP_BALANCES_DELAY = 35 * 12`) between successive calls.

contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::snapBalances()

```
function snapBalances() external {
    if (consolidationCount > 0 && msg.sender != validatorRegistrator) {
        revert("Consolidation in progress");
    }

    targetStrategy.snapBalances();
}
```

contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::snapBalances()

```
function snapBalances() external onlyRegistrator {
    uint64 currentTimestamp = SafeCast.toUint64(block.timestamp);
    require(
        snappedBalance.timestamp + SNAP_BALANCES_DELAY < currentTimestamp, // @audit reverts if called within 420 seconds of the
            ↪ last snap
        "Snap too soon"
    );

    // ... snip ...
}
```

The `requestConsolidation()` function calls `targetStrategy.snapBalances()` at the end of its execution. If an attacker has already called `snapBalances()` within the preceding 420 seconds, this internal call reverts with "Snap too soon", causing the entire `requestConsolidation()` transaction to fail.

contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::requestConsolidation()

```
function requestConsolidation(
    address _sourceStrategy,
    bytes[] calldata sourcePubKeys,
    bytes calldata targetPubKey
) external payable onlyOwner {
    // ... snip ...

    // Snap the balances for the last time on the new Compounding Staking Strategy
    // until the consolidations are confirmed
    targetStrategy.snapBalances(); // @audit reverts if snapBalances() was called within the last 420 seconds
}
```

An attacker can sustain this attack indefinitely by calling `ConsolidationController.snapBalances()` every 420 seconds. The cost of sustaining the attack is minimal as `snapBalances()` is a relatively cheap operation. While the owner could wait for the cooldown to expire, the attacker can monitor the mempool and frontrun the owner's `requestConsolidation()` call with another `snapBalances()` call, resetting the cooldown window.

This issue has a medium impact as an attacker can sustain the DoS indefinitely at minimal gas cost, completely blocking the consolidation process which is core to the migration of validators from old staking strategies to the new compounding strategy.

This issue has a medium likelihood as the attack is permissionless and requires only monitoring the mempool for `requestConsolidation()` transactions, though the owner could attempt to mitigate it by using private mempools.

Recommendations

Separate the `CompoundingValidatorManager.snapBalances()` call from `requestConsolidation()` so that the owner can snap balances independently before initiating the consolidation, or allow `requestConsolidation()` to skip the snap if one was already taken within an acceptable window.

Alternatively, allow the registrator or owner to pause `ConsolidationController.snapBalances()` before calling `requestConsolidation()`, preventing external callers from triggering the cooldown during the window when a consolidation is about to be initiated.

Resolution

The development team has implemented the following changes:

1. Updated `ConsolidationController.requestConsolidation()` to skip the snap if one was already taken within the last 420 seconds.
2. Updated `ConsolidationController.verifyBalances()` to allow for verification of balances while consolidations are in progress for snaps taken before the consolidation process started. This prevents an edge case where balances in the Compounding Staking Strategy are left stale for too long if they're not verified before the consolidation process starts.

This issue has been resolved in PR #2820.

OGVC-04	Excess Fee Payment Misaccounted As Strategy Balance		
Assets	contracts/contracts/strategies/NativeStaking/ConsolidationController.sol contracts/contracts/strategies/NativeStaking/ValidatorRegistrator.sol contracts/contracts/strategies/NativeStaking/CompoundingValidatorManager.sol		
Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

Excess ETH from overpaid system contract fees in `requestConsolidation()` and `validatorWithdrawal()` is left in the respective strategy contracts and subsequently misaccounted as legitimate validator balance.

The `ConsolidationController.requestConsolidation()` function forwards the entire `msg.value` to the source strategy:

contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::requestConsolidation()

```
ValidatorAccountant(_sourceStrategy).requestConsolidation{
    value: msg.value
}(sourcePubKeys, targetPubKey);
```

Inside `ValidatorRegistrator.requestConsolidation()`, the `BeaconConsolidation` library is called once per source validator. Each call fetches the current fee from the EIP-7251 consolidation request contract and forwards only that exact amount:

contracts/contracts/beacon/BeaconConsolidation.sol::request()

```
function request(bytes calldata source, bytes calldata target)
    internal
    returns (uint256 fee_)
{
    // ... snip ...
    fee_ = fee();

    // @audit only fee_ is sent, not msg.value
    (bool success, ) = CONSOLIDATION_REQUEST_ADDRESS.call{ value: fee_ }(
        abi.encodePacked(source, target)
    );

    require(success, "Consolidation request failed");
}
```

The total fees consumed equal `fee * sourcePubKeys.length`. Any excess (`msg.value - totalFees`) remains in the source strategy contract. When `ValidatorAccountant._doAccounting()` next executes, this excess is included in the `ethRemaining` calculation:

contracts/contracts/strategies/NativeStaking/ValidatorAccountant.sol::_doAccounting()

```
uint256 ethRemaining = address(this).balance - consensusRewards;
// ... snip ...
if (ethRemaining == 0) {
    return accountingValid;
} else if (ethRemaining < fuseIntervalStart) {
    // @audit excess fee payment counted as consensus rewards
    consensusRewards += ethRemaining;
```

The same pattern exists in `ConsolidationController.validatorWithdrawal()`, which forwards `msg.value` to the target strategy:

contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::validatorWithdrawal()

```
targetStrategy.validatorWithdrawal{ value: msg.value }(
    publicKey,
    amountGwei
);
```

Inside `CompoundingValidatorManager.validatorWithdrawal()`, the `PartialWithdrawal` library similarly consumes only the exact fee. Excess ETH remains in the target strategy and is captured in the next `snapBalances()` call via `address(this).balance`, inflating `lastVerifiedEthBalance` through `verifyBalances()`.

The caller must overestimate `msg.value` because the fee is dynamic and calculated on-demand by the system contract. This is further exacerbated by the fact that the registrator is expected to be a multisig, introducing a delay between transaction creation and execution during which the fee can change. Underestimating the fee causes the transaction to revert, so overpayment is the practical default.

This issue has a low impact as the excess amounts are likely small (the difference between overestimated and actual fees), and the misaccounting inflates strategy balances rather than causing direct fund loss. However, if the excess is large enough to fall within the fuse interval in `ValidatorAccountant._doAccounting()`, it could blow the accounting fuse and require manual intervention from the strategist.

This issue has a medium likelihood as the registrator must consistently overpay to avoid reverts, making excess ETH residuals a near-certainty for every consolidation and withdrawal request.

Recommendations

Refund excess ETH back to the caller after fee payment. Track the total fees consumed within `requestConsolidation()` and `validatorWithdrawal()`, then return the difference between `msg.value` and the total fees to `msg.sender` at the end of each function.

Resolution

The development team has acknowledged this issue with the following comment:

> *"Although the consolidation fee can spike to the millions of wei, it's still a tiny amount in ETH terms which is to 18 decimals. Tripping the fuse will require the validators to be swept and then many ETH added in overpayment. The chances of this happening become even smaller once the number of validators drop from the consolidations."*

OGVC-05	`MIN_CONSOLIDATION_PERIOD` Underestimates Minimum Consolidation Time
Assets	`contracts/contracts/strategies/NativeStaking/ConsolidationController.sol`
Status	**Resolved:** See Resolution
Rating	Informational

Description

The `MIN_CONSOLIDATION_PERIOD` constant underestimates the actual minimum time from a consolidation request to its completion on the beacon chain by 5 epochs (~32 minutes).

The constant is derived from `MIN_VALIDATOR_WITHDRAWABILITY_DELAY` (256 epochs).

contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::MIN_CONSOLIDATION_PERIOD

```
/// @dev Minimum time that must pass before a consolidation request can be processed.
/// 256 epochs * 32 slots/epoch * 12 seconds/slot = 98304 seconds (~27 hours)
/// The actual time can be a lot longer than this depending on the number of
/// requests in the beacon chain's pending consolidation queue.
uint256 internal constant MIN_CONSOLIDATION_PERIOD = 256 * 32 * 12;
```

However, the beacon chain's `process_consolidation_request()` sets the source validator's `exit_epoch` via `compute_consolidation_epoch_and_update_churn()`, which in turn calls `compute_activation_exit_epoch()`. This function adds `MAX_SEED_LOOKAHEAD + 1` (5 epochs) to the current epoch:

ethereum/consensus-specs/specs/phase0/beacon-chain.md

```
def compute_activation_exit_epoch(epoch: Epoch) -> Epoch:
    return Epoch(epoch + 1 + MAX_SEED_LOOKAHEAD)  # @audit MAX_SEED_LOOKAHEAD = 4
```

The `withdrawable_epoch` is then set to `exit_epoch + MIN_VALIDATOR_WITHDRAWABILITY_DELAY`, giving a minimum of `current_epoch + 5 + 256 = current_epoch + 261` epochs. Since `process_pending_consolidations()` only processes entries where `source_validator.withdrawable_epoch <= next_epoch`, the true protocol minimum is 261 epochs, not 256 epochs.

This is rated as informational as the constant is used as a lower-bound guard in `confirmConsolidation()` and the existing comment already notes the actual time will be longer. The `ConsolidationController` owner still needs to verify that the snapshot was taken after all pending consolidations have been processed on the beacon chain.

Recommendations

Consider tightening the constant to `261 * 32 * 12` and updating the accompanying comment to reflect the additional 5 epochs from `compute_activation_exit_epoch()`.

Resolution

The development team has updated the constant to `261 * 32 * 12` and the accompanying comment to reflect the additional 5 epochs from `compute_activation_exit_epoch()`.

This issue has been resolved in PR #2821.

OGVC-06	Missing `sourcePubKeys` Input Validation In `requestConsolidation()`
Assets	`contracts/contracts/strategies/NativeStaking/ConsolidationController.sol`
Status	**Resolved:** See Resolution
Rating	Informational

Description

The `requestConsolidation()` function in `ConsolidationController` does not validate that `sourcePubKeys` is non-empty, allowing the owner to initiate a consolidation with zero source validators and leaving the contract in an inconsistent state.

The `requestConsolidation()` function stores consolidation state before forwarding the call to the source strategy. The `consolidationCount` is set to `sourcePubKeys.length`, which is zero when an empty array is passed. However, the other state variables (`consolidationStartTimestamp`, `sourceStrategy`, and `targetPubKeyHash`) are all set to non-zero values regardless.

contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::requestConsolidation()

```
function requestConsolidation(
    address _sourceStrategy,
    bytes[] calldata sourcePubKeys,
    bytes calldata targetPubKey
) external payable onlyOwner {
    require(consolidationCount == 0, "Consolidation in progress");
    _checkSourceStrategy(_sourceStrategy);

    bytes32 targetPubKeyHashMem = _hashPubKey(targetPubKey);
    // ... target validator checks ...

    // @audit sourcePubKeys.length is not checked > 0
    consolidationCount = SafeCast.toUint64(sourcePubKeys.length);
    consolidationStartTimestamp = uint64(block.timestamp);
    sourceStrategy = _sourceStrategy;
    targetPubKeyHash = targetPubKeyHashMem;

    ValidatorAccountant(_sourceStrategy).requestConsolidation{
        value: msg.value
    }(sourcePubKeys, targetPubKey);

    targetStrategy.snapBalances();
}
```

After this call completes with an empty array, `consolidationCount` remains zero while `targetPubKeyHash` holds a stale value. This stale `targetPubKeyHash` causes the `stakeEth()` function to reject deposits to the target validator, since it checks `_hashPubKey(validatorStakeData.pubkey) != targetPubKeyHash`. Neither `confirmConsolidation()` nor `failConsolidation()` can be used to reset this state as both require `consolidationCount > 0`. The only recovery path is to execute a full consolidation cycle with non-empty source validators to overwrite the stale state.

This issue is rated as informational as `requestConsolidation()` is restricted to `onlyOwner`, making an accidental call with an empty array unlikely. The stale state is recoverable, and no funds are at risk.

Recommendations

Add a zero length check on `sourcePubKeys` at the beginning of `requestConsolidation()` to prevent the contract from entering an inconsistent state.

Resolution

The development team has added a zero length check on `sourcePubKeys` at the beginning of `requestConsolidation()`.

This issue has been resolved in PR #2822.

OGVC-07	`snapBalances()` Lacks `MIN_CONSOLIDATION_PERIOD` Check
Assets	`contracts/contracts/strategies/NativeStaking/ConsolidationController.sol`
Status	**Resolved:** See Resolution
Rating	Informational

Description

The `ConsolidationController.snapBalances()` function does not enforce the `MIN_CONSOLIDATION_PERIOD` timing constraint during an active consolidation, unlike `confirmConsolidation()` which validates this condition before proceeding.

The `confirmConsolidation()` function checks that the balance snapshot was taken after the minimum consolidation period has elapsed. However, `snapBalances()` itself does not enforce any timing restriction relative to the consolidation start, allowing the registrator to snap balances prematurely during an active consolidation.

contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::snapBalances()

```
function snapBalances() external {
    if (consolidationCount > 0 && msg.sender != validatorRegistrator) {
        revert("Consolidation in progress");
    }
    // @audit no check that block.timestamp > consolidationStartTimestamp + MIN_CONSOLIDATION_PERIOD

    targetStrategy.snapBalances();
}
```

A premature snap overwrites the `snappedBalance` on the target strategy with a timestamp that predates the consolidation period, causing `confirmConsolidation()` to revert with "Source not withdrawable" until the registrator calls `snapBalances()` again after the period has elapsed. Adding the same timing check to `snapBalances()` would prevent unnecessary premature snaps and provide defence in depth for the consolidation timing invariant.

This issue is rated as informational as the existing check in `confirmConsolidation()` prevents any actual misuse and the registrator is a trusted role.

Recommendations

Consider adding a `MIN_CONSOLIDATION_PERIOD` check to `snapBalances()` when a consolidation is in progress, ensuring the function reverts if `block.timestamp <= consolidationStartTimestamp + MIN_CONSOLIDATION_PERIOD`.

Resolution

The development team has added a `MIN_CONSOLIDATION_PERIOD` check to `snapBalances()` when a consolidation is in progress.

This issue has been resolved in PR #2826.

OGVC-08	Miscellaneous General Comments
Assets	/*
Status	**Resolved:** See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. **Incorrect NatSpec For `stakingMonitor`**

   Related Asset(s): contracts/contracts/strategies/NativeStaking/ValidatorRegistrator.sol

   The NatSpec comment for `stakingMonitor` incorrectly states the account is allowed to modify `stakeETHThreshold`, when this is actually managed by the governor via the `onlyGovernor`-protected `setStakeETHThreshold()` function. The `stakingMonitor` can only reset `stakeETHTally`.

   contracts/contracts/strategies/NativeStaking/ValidatorRegistrator.sol::stakingMonitor

   ```
   /// @notice The account that is allowed to modify stakeETHThreshold and reset stakeETHTally
   // @audit stakeETHThreshold is set by the governor, not the staking monitor
   address public stakingMonitor;
   ```

   Update the NatSpec comment to accurately reflect the role of `stakingMonitor`, stating that it is the account allowed to reset `stakeETHTally`.

2. **Over-Restrictive Consolidation Check In `removeSsvValidator()`**

   Related Asset(s): contracts/contracts/strategies/NativeStaking/ConsolidationController.sol

   The `removeSsvValidator()` function in `ConsolidationController` unnecessarily blocks SSV validator removal from all source strategies during consolidation, even when the removal targets a strategy unrelated to the ongoing consolidation.

   The `removeSsvValidator()` function checks `consolidationCount == 0`, which prevents removal from both source strategies. Since consolidation only involves one source strategy at a time (tracked by `sourceStrategy`), validators in the other strategy should still be removable. This is inconsistent with `exitSsvValidator()`, which permits exits from any source strategy during consolidation.

   contracts/contracts/strategies/NativeStaking/ConsolidationController.sol::removeSsvValidator()

   ```
   function removeSsvValidator(
       // ... snip ...
   ) external onlyRegistrator {
       _checkSourceStrategy(_sourceStrategy);
       // @audit This blocks removal from ALL strategies, not just the one being consolidated
       require(consolidationCount == 0, "Consolidation in progress");

       ValidatorAccountant(_sourceStrategy).removeSsvValidator(
           publicKey,
           operatorIds,
           cluster
       );
   }
   ```

   Replace the blanket `consolidationCount == 0` check with a targeted check that only prevents removal from the source strategy currently undergoing consolidation. Checking `_sourceStrategy != sourceStrategy` is sufficient, as `sourceStrategy` is `address(0)` when no consolidation is in progress, which will always differ from a valid source strategy address.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team's responses to the raised issues above are as follows.

1. Updated the NatSpec comment for `stakingMonitor` to accurately reflect the role of `stakingMonitor`, stating that it is the account allowed to reset `stakeETHTally`.
2. Updated the `removeSsvValidator()` function to only prevent removal from the source strategy currently undergoing consolidation.

These issues have been resolved in PR #2823.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurance. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	Likelihood: Low	Likelihood: Medium	Likelihood: High
High	Medium	High	Critical
Medium	Low	Medium	High
Low	Low	Low	Medium

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'